

Chess Neural Network

Caleb Faucher

University of Nebraska at Omaha

CSCI8598

Dr. Zhong

May 1st, 2026

The goal of this project is to explore how a computer can learn to play chess using deep learning techniques combined with search algorithms. Chess has long been considered a challenging problem in artificial intelligence because it requires both strategic planning and the ability to evaluate complex positions. In this paper, I investigate how a neural network can be trained through self-play to make decisions in chess, and how its performance can be improved by integrating it with a search method. The main problem addressed is how to enable a model to learn effective gameplay without relying on handcrafted rules or large datasets of expert games. If successful, such a system could have broader applications in areas where decision-making under uncertainty is required, such as robotics, logistics, and optimization problems.

The development of chess-playing artificial intelligence has evolved significantly over time. Traditional chess engines, such as Stockfish, rely heavily on brute-force search combined with carefully designed evaluation functions created by human experts. These systems are extremely powerful but depend on domain-specific knowledge. A major breakthrough in the field came with systems like AlphaZero, which demonstrated that a neural network could learn to play chess at a very high level through self-play alone. AlphaZero introduced a method that combines a policy and value network with Monte Carlo Tree Search, allowing the system to both evaluate positions and explore possible future moves. Compared to these advanced systems, my approach is much simpler and uses a smaller neural network and fewer computational resources. However, it follows the same core idea of combining learning with search, making it a useful educational approximation of more complex methods.

The methodology used in this project consists of two main components: a neural network that evaluates chess positions and a Monte Carlo Tree Search algorithm that uses these evaluations to guide decision-making. The chessboard is represented as a three-dimensional tensor with twelve channels, where each channel corresponds to a different type of piece for each player. This representation allows the neural network to process spatial information effectively using convolutional layers. The model itself is implemented using PyTorch and includes convolutional layers for feature extraction followed by fully connected layers. The network produces two outputs: a policy, which represents a probability distribution over possible moves, and a value, which estimates the likelihood of winning from the current position.

To improve decision-making, the model is combined with Monte Carlo Tree Search. Instead of selecting moves directly from the neural network's output, the algorithm simulates many possible future game states and uses the network to evaluate them. During the selection phase, nodes are chosen according to a balance between exploration and exploitation, guided by the following formula:

$$U(s, a) = Q(s, a) + c_{puct} P(s, a) \sqrt{N(s)} \quad 1 + N(s, a) \quad U(s, a) = Q(s, a) + c_{puct} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)} \quad U(s, a) = Q(s, a) + c_{puct} P(s, a)$$

This process allows the system to prioritize moves that are both promising according to the network and insufficiently explored. The search tree is expanded by adding new nodes corresponding to legal moves, and the neural network provides value estimates for these positions. These values are then propagated back through the tree to update the statistics of previously visited nodes. Through repeated simulations, the algorithm identifies moves that are likely to lead to better outcomes.

The experiments conducted in this project rely entirely on self-generated data. Instead of using an external dataset, the model plays games against itself using the python-chess, which provides the rules and mechanics of the game. Each game produces a sequence of board states, moves, and final outcomes, which are then used to train the neural network. The training process was carried out over several epochs, with a small number of games per epoch due to computational limitations. I compared the behavior of the system before and after integrating Monte Carlo Tree Search and observed how its performance changed over time.

The results show a clear difference between the model's behavior with and without search. Without Monte Carlo Tree Search, the model tends to select moves that appear almost random, as the neural network alone is not sufficiently trained to provide meaningful guidance. After adding the search component, the system begins to demonstrate more structured behavior, such as making sensible captures and avoiding immediate blunders. Over successive training epochs, the loss values decrease, and the gameplay becomes more consistent. Although the model remains far from competitive with strong chess engines, it shows measurable improvement and begins to exhibit basic strategic understanding. Playing against the model also revealed that later versions were more stable and less prone to obvious mistakes than earlier ones.

In conclusion, this project demonstrates that even a relatively simple implementation of a neural network combined with Monte Carlo Tree Search can learn to play chess at a basic level through self-play. The approach highlights the importance of combining learning-based evaluation with search-based planning. While the system developed here is limited by its small scale and computational constraints, it successfully captures the fundamental ideas behind more advanced systems. Future improvements could include increasing the size of the network, training on more games, and incorporating additional techniques such as replay buffers or more sophisticated exploration strategies. Overall, this work provides a foundation for understanding how modern artificial intelligence systems can learn complex decision-making tasks without explicit human guidance.